

6. 存储器

- 6.1 存储器概述
- 6.2 随机存储器
- 6.3 数据校验码
- 6.4 Cache高速缓存
- 6.5 虚拟存储器
- 6.6 RAID

6.1 存储器概述

- 存储器的分类
 - 按在计算机中的作用分类
 - 按存储介质分类
 - 按存储方式分类
 - 按信息的可保存性分类
- 存储器的性能指标
- 存储体系的设计

存储器的分类

按在计算机中的作用

■ 寄存器型存储器:

- 由寄存器构成, 封装在CPU内
- 用于存放当前正在执行的指令和使用的数据
- 用触发器实现, 速度快, 容量小 $O(KB)$

■ 高速缓冲存储器(Cache)

- 位于CPU内部或位于主存和CPU之间
- 用来存放当前正在执行的局部程序段和数据
- 用SRAM实现, 速度 $O(ns)$ 、容量 $O(MB)$

■ 主存储器:

- 位于CPU之外
- 用来存放已被启动的程序及所用数据
- 用DRAM实现, 速度 $O(ns)$ 、容量 $O(GB)$

■ 辅助存储器 (外存)

- 不能由CPU的指令直接访问
- 用来存放暂不运行的程序、数据或存档文件
- 用磁、光、flash半导体等存储器件实现, 容量大, 但速度慢 $O(ms)$

■ 其它功能的存储器

- 控制存储器: 存储微程序代码, 只读
- 表格存储器: 为加快CPU处理而设置 (倒数表、函数表)
- 字库和数据缓冲存储器: 显示和印刷输出设备中
-

存储器的分类

按存储介质

■ 半导体存储器(SCM)

- 速度快，用作内存，ROM，RAM
- 记忆原理：双稳态触发器、电容等

■ 磁表面存储器MSM

- 用陶瓷、非磁性金属或塑料作载磁体，磁化后具有两种不同的剩磁状态记录信息“1”和“0”
- 非易失(non-volatile)，容量大并且每位价格低，用作外存

■ 光盘存储器ODM

- 以光信息做为存储的载体并用来存储数据
- 非易失，可靠性高，保存时间长，容量大且易于更换
- 存储速度比硬盘低一个数量级

■ 新型存储器：铁电存储器FeM(FRAM)、相变存储器PCM、阻变存储器ReRAM、.....

- 非易失，速度快，低功耗，体积小

存储器的分类

按存储方式

■ 随机访问存储器(RAM) *SRAM, DRAM*

- 存储器任意单元可随时访问且访问所需时间相同
- 访问时间与存储单元所处的物理位置无关
- 速度快(ns)
- 主存和Cache

■ 只读存储器(ROM)

- 正常工作时只读，能随机读出，不能随机写入
- PROM：一次写
- 可多次改写ROM：EPROM、E²PROM *Flash*

■ 相联存储器(CAM)

- 按内容检索到存储位置，进行读写
- 速度快(ns)，价格高
- 可以实现快速地查找快表，适合于信息的检索和更新

■ 顺序存储器(SAS)

- 存储时以数据块为单位存储，顺序地记录在存储介质上
- 数据按顺序从存储载体的始端读出或写入，存取时间的长短与数据所在位置有关
- 速度慢、容量大、成本低，用作后援外存
- 磁带、电荷耦合器件CCD、VCD

■ 直接存取存储器DAS

- 信息的组织同SAS，介于随机和顺序存取之间
- 可以直接定位到要读写的数据块，存取时间的长短与数据所在位置有关
- 速度慢(ms)
- 磁盘

存储器的分类

按信息的可保存性：断电后是否丢失数据

- **易失性存储器 (VM)：** 断电后，信息即丢失
 - **破坏性存储器：** DRAM
 - 特点：读出时原存信息被破坏，需重写
 - **非破坏性存储器：** SRAM
 - 特点：读出时，原存信息不被破坏
- **非易失性/永久性存储器 (NVM)**
 - 特点：断电后，信息不丢失
 - ROM、闪存、磁盘

存储器的性能指标

■ 1.存储容量

- 所能容纳的二进制位(bit)个数的总和，即构成内存的存储位元的总和
- 存储容量 = 存储单元个数 × 每个存储单元的存储位元个数
- 常用单位：B(Byte)、KB(2^{10})、MB(2^{20})、GB(2^{30})、TB(2^{40})

存储器的性能指标

■ 2. 存取速度

■ 存取时间 T_A ：启动一次存储器操作到完成该操作所用时间

- 亦称访问时间，是反映存储器速度的指标，决定了CPU发出读/写命令后必须等待的时间

- 读：从启动读命令到读出的数据送到寄存器所需的时间

- 写：从启动到将寄存器内容写入指定存储单元的时间

■ 存储周期 T_S ：连续两次启动同一存储器进行存取操作所需的最小时间间隔，亦称存取周期或访问周期

- 两次启动包括：两次读，两次写，一次读或一次写

■ $T_A < T_S$ ： T_A 、 T_S 的差别依赖于存储信息的器件和电路，存储介质和控制线路需要恢复时间

- 破坏性读出须重写；非破坏性读出不须重写

存储器的性能指标

■ 3. 传输性能

- **存取宽度**：一次访存操作可存取的数据位数或字节数，
 - 存取宽度由编址方式决定
- **存储带宽**：每秒传输的最大数据量(位/秒)
 - 改善计算机性能瓶颈的一个关键优化因素
 - 例：存取周期为500ns，每个存取周期可访问16位，则带宽：
 - $16\text{位} * (1/500\text{ns}) = 32\text{M位/秒}$

存储器的性能指标

- **4. 可靠性：** 用MTBF(Mean Time Between Failure)来衡量主存的可靠性
 - MTBF表示两次故障之间的平均时间间隔，越长可靠性越高
 - 内存常采用纠错编码技术进行容错处理，延长MTBF
 - 例：银河-2，CPU字长64位、存储器字长72位，采用海明码“纠1检2”，提高主存可靠性。

存储器的性能指标

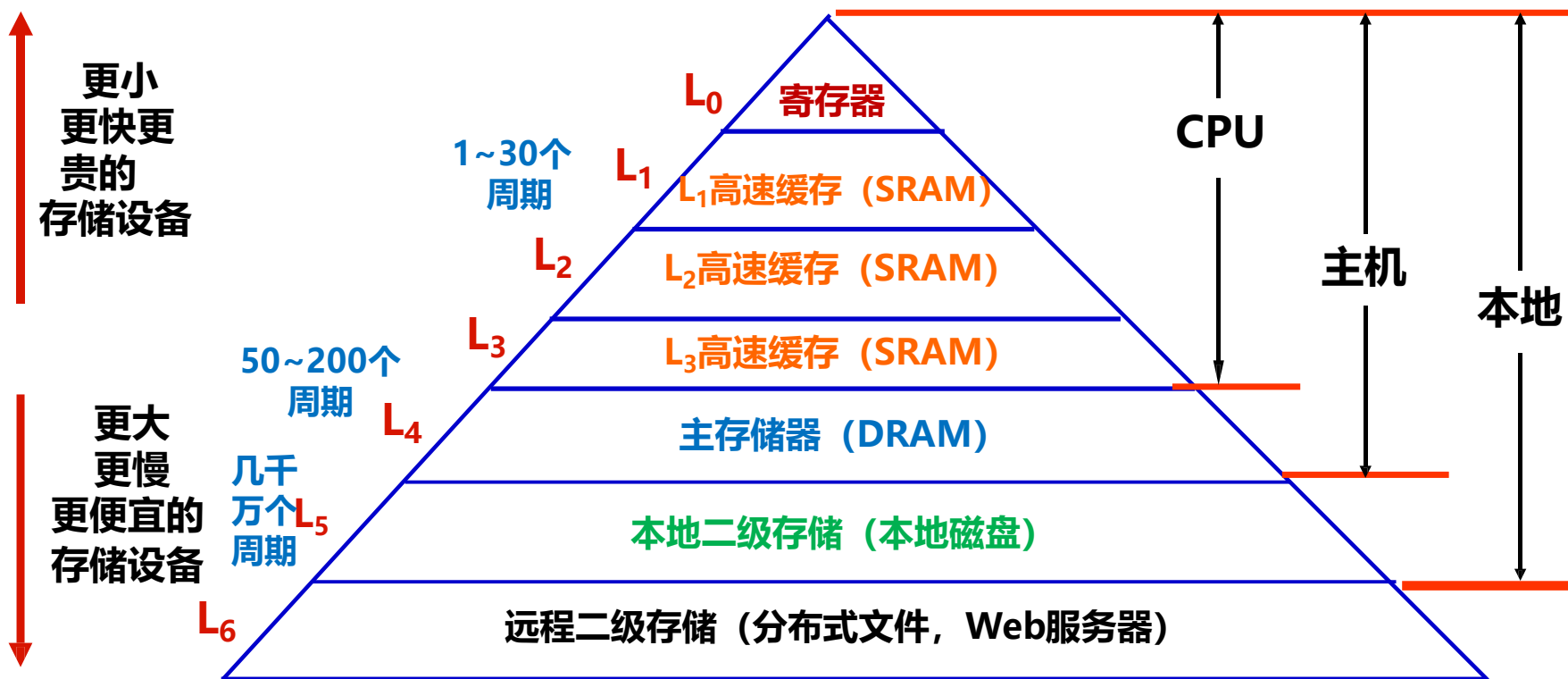
■ 5. 功耗和集成度

- 存储带宽功耗(Power Loss): 反映存储器件耗电多少(单位: mw/片、w/存储器)
- 维持功耗: 保持时功耗(几十~几百mw/片)
- 工作功耗: 读写时功耗(500~1500mw/片)
- 集成度(Integration Level): 单位存储芯片的存储元件个数

存储体系的设计

■ 存储的层次结构

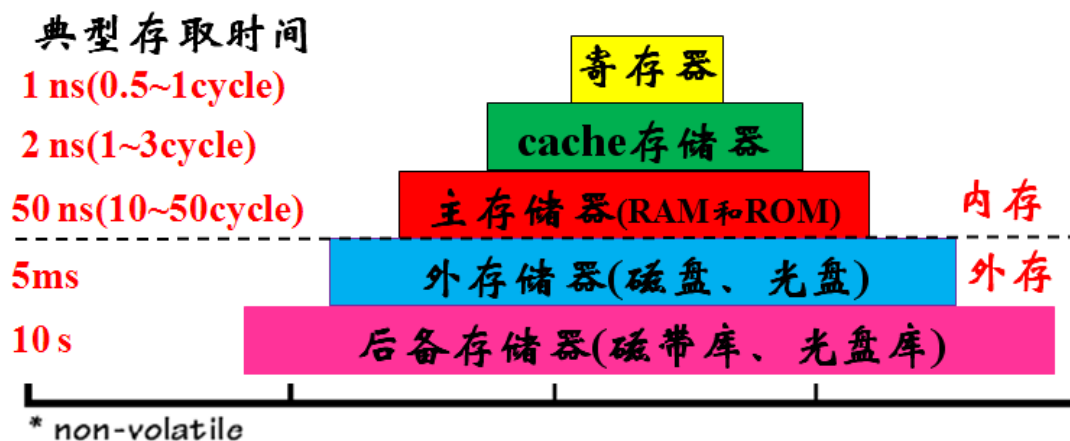
- **综合性指标——性价比**：更高的容量、速度，更低的位价格
- 为提高性价比，将各存储器组成一个金字塔式的层次结构，取长补短协调工作



存储体系的设计

	Capacity	Latency	Cost
Register	<1KB	1ns	\$\$\$\$
SRAM	1MB	2ns	\$\$\$
DRAM	1GB	10ns	\$
Hard disk*	100GB	10ms	¢
Want	100GB	1ns	cheap

* non-volatile



存储体系的设计

内存与外存的比较

外存储器

- 存取速度慢
- 成本低、容量很大
- CPU不能直接访问（外存中的程序及数据必须先传送到内存，才能被 CPU 使用）
- 非易失性存储器(Nonvolatile)，用于长久存放信息

内存存储器

- 存取速度快
- 成本高、容量相对较小
- CPU可以直接对内存中的指令及数据进行读、写操作
- 易失性存储器(volatile)，用于临时存放正在运行的程序和数据

存储体系的设计

- CPU 运行时，需要的操作数大部分来自寄存器
- 需从(向) 存储器中取(存)数时，先访问cache，若在，取自cache；若不在cache，则访问RAM，若在，则取自RAM；若不在RAM，则访问磁盘，操作数从磁盘中读出→RAM →cache
- 离CPU较近的数据是较远的那些层次的子集，最底层存放着所有数据。离处理器越远的存储器访问时间越长。
- 数据总是在相邻两层之间复制传送

存储体系的设计

■ 程序访问的局部性：

- 分析大量典型程序的运行情况，结果表明：在较短时间间隔内，程序产生的地址/所访问数据往往集中在存储器的一个很小范围内。
- **时间局部性**(Temporal Locality)：刚被访问过的存储单元很可能不久又被访问
 - 让最近被访问过的信息保留在靠近CPU的存储器中
- **空间局部性** (Spatial Locality)：刚被访问过的存储单元的邻近单元很可能不久被访问
 - 将刚被访问过的存储单元的邻近单元调到靠近CPU的存储器中
- 为什么会具有局部性？
 - 指令：按序存放，地址连续。如循环程序段等重复执行
 - 数据：连续存放。如数组元素重复、按序访问

程序局部性示例

程序A和B中：

- 对数组A引用的局部性？
- 变量sum的局部性如何？
- 对于指令，for循环体的局部性如何？

程序段A：

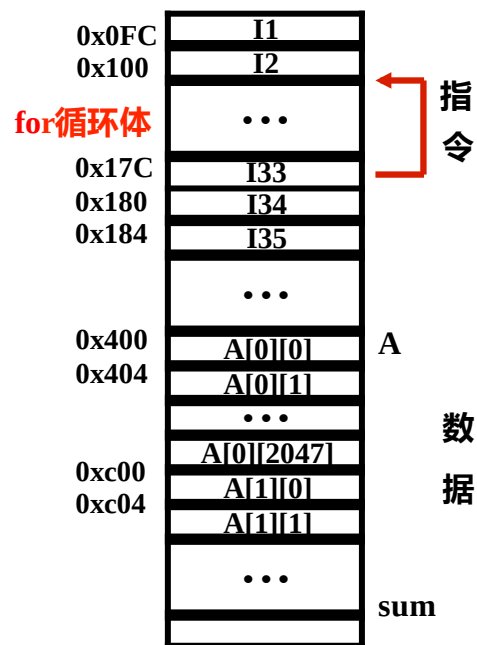
```
int sumarrayrows(int A[M][N])
{
    int i, j, sum=0;
    for (i=0; i<M, i++)
        for (j=0; j<N, j++)
            sum+=A[i][j];
    return sum;
}
```

程序段B：

```
int sumarraycols(int A[M][N])
{
    int i, j, sum=0;
    for (j=0; j<N, j++)
        for (i=0; i<M, i++)
            sum+=A[i][j];
    return sum;
}
```

假定数组在存储器中按行优先顺序存放

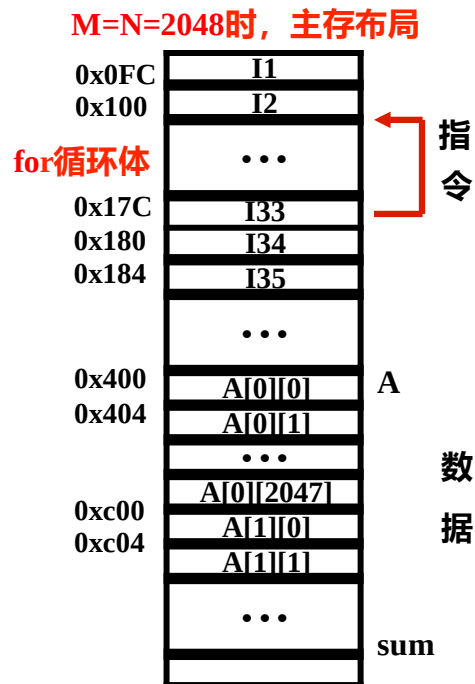
M=N=2048时，主存布局



程序局部性示例

```
程序段A:  
int sumarrayrows(int A[M][N])  
{  
    int i, j, sum=0;  
    for (i=0; i<M, i++)  
        for (j=0; j<N, j++)  
            sum+=A[i][j];  
    return sum;  
}
```

- ◆ **数组A**: 访问顺序为A[0][0], A[0][1], ..., A[0][2047]; A[1][0], A[1][1], ..., A[1][2047], 与存放顺序一致, 空间局部性好! 而每个A[i][j]只被访问一次, 故**时间局部性差!**
- ◆ **变量sum**: 单个变量不考虑空间局部性; 每次循环都访问sum, 故其**时间局部性好!**
- ◆ **for循环体**: 循环体内指令按序连续存放, **空间局部性好!** 循环体被连续重复执行2048×2048次, 故**时间局部性好!**

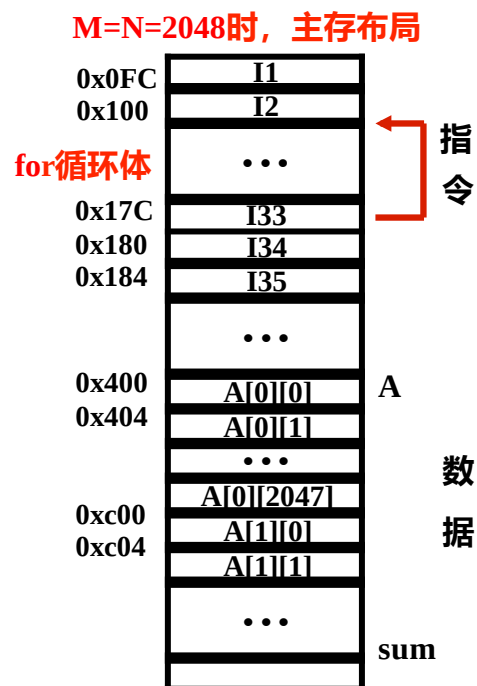


程序局部性示例

程序段B:

```
int sumarraycols(int A[M][N])
{
    int i, j, sum=0;
    for (j=0; j<N, j++)
        for (i=0; i<M, i++)
            sum+=A[i][j];
    return sum;
}
```

- 数组A: 访问顺序为A[0][0], A[1][0], ..., A[2047][0]; A[0][1], A[1][1], ..., A[2047][1];... 与存放顺序不一致, 每次跳过2048个单元, 若CPU与Cache数据交换单位小于2KB, 则**没有空间局部性! 时间局部性也差**
- 变量sum: (同程序A)
- for循环体: (同程序A)



程序局部性示例

程序段A:

```
int sumarrayrows(int A[M][N])
{
    int i, j, sum=0;
    for (i=0; i<M, i++) for
        (j=0; j<N, j++)
            sum+=A[i][j];
    return sum;
}
```

程序段B:

```
int sumarraycols(int A[M][N])
{
    int i, j, sum=0;
    for (j=0; j<N, j++)
        for (i=0; i<M, i++)
            sum+=A[i][j];
    return sum;
}
```

实际运行结果

(2GHz Intel Pentium 4)

程序A: 59,393,288 时钟周期

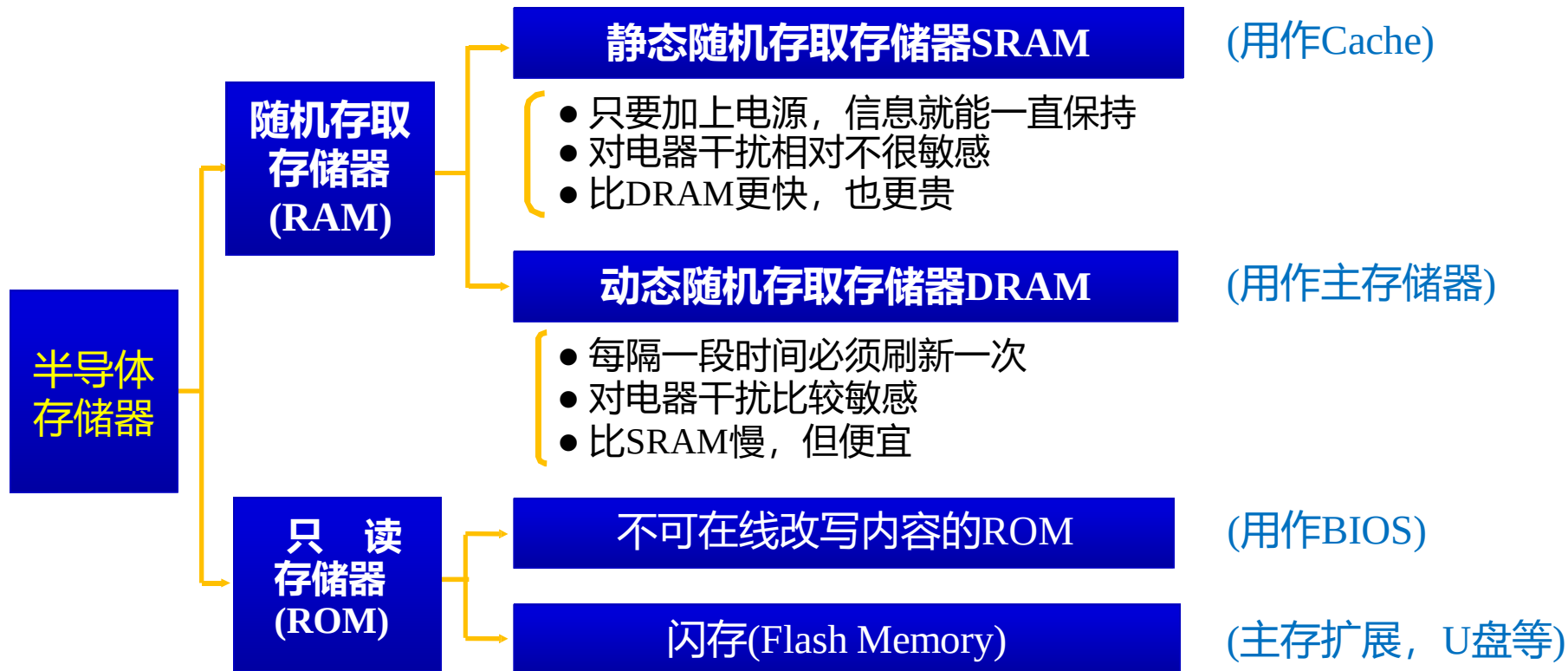
程序B: 1,277,877,876 时钟周期

程序A比B快21.5 倍!!

在评估机器性能时，存储器系统性能是一个重要指标。利用存储器的层次结构能够提高性能，不能把存储器看作是一个平面的随机访问存储设备，必须理解存储层次结构如何工作才能系统达到良好的性能。

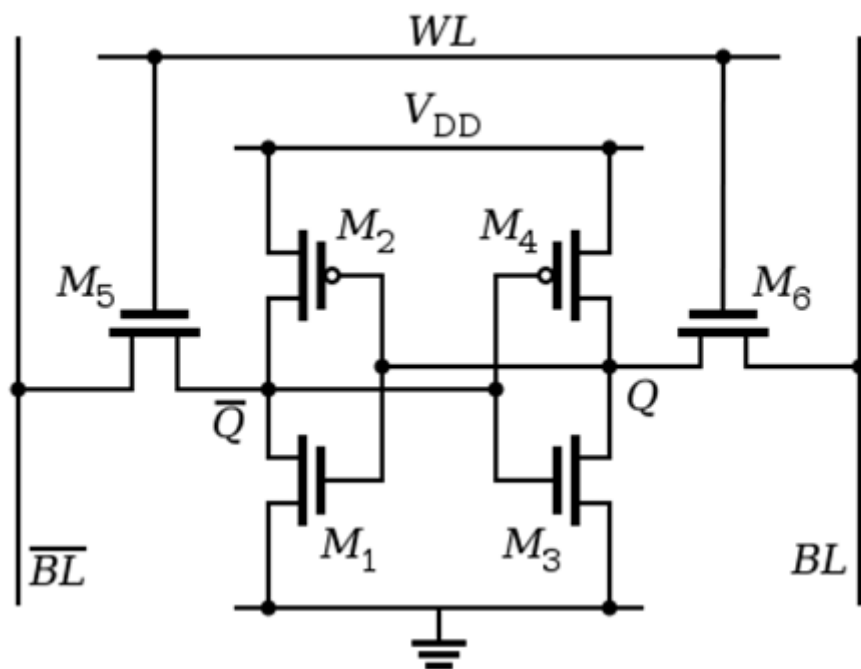
6.2 随机存取存储器

- 内存由半导体存储器芯片组成，芯片有多种类型
- 随机存取存储器（RAM）：SRAM & DRAM



SRAM

- SRAM的基本存储单元： 看作双稳态触发器



SRAM数据保存在触发器中，只要供电，数据就一直保持，不需要重写数据来保持数据不变。

空闲时：

- 字线低电平，M5与M6处于断路，把基本单元与位线隔离。
- 由M1 - M4组成的触发器保持其状态。

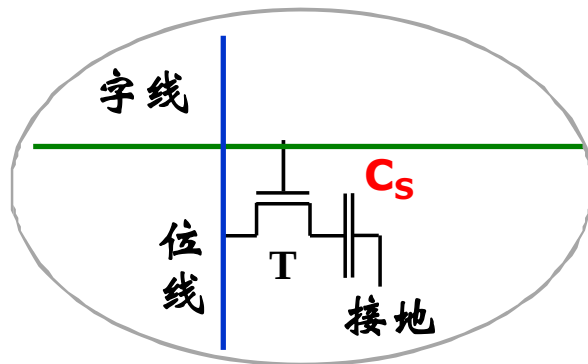
写入时：

- 字线为高电平，栅极导通
- 位线上是被写入的I/O端信息0或1
- 存储单元按位线的状态被置为0或1

读出时：

- 字线为高电平，栅极导通
- 根据存储单元的状态改变位线的输出电平，读出到I/O端

DRAM



■ 存储单元的基本原理

- 数据记忆在电容CS上，T为门控管，控制数据进出
- 栅极接读/写选择线(字线)，漏和源分别接数据线(位线)和记忆电容CS
- 存储数据“1”或“0”：根据电容CS上电荷量的有无来判别

■ 读写原理：在选择(字)线上加高电平，使T管导通

- 写“0”时，位线上加低电平，使CS上电荷对数据线放电
- 写“1”时，位线上加高电平，使数据线对CS充电
- 读出时，根据位线（数据线）上是否有电流，区分读出的是“1”还是“0”

DRAM

- 优点

- 电路元件少，工作功耗小，集成度高，广泛用于大容量主存储器中

- 缺点

- 速度慢、数据以电荷形式保存在电容中
 - 电容漏电使得电荷通常只能维持2ms左右，因此要定期在2ms内刷新(读出后重新写回)

SRAM 与 DRAM

- 一个DRAM存储单元大约需要一个晶体管和一个电容（不包括行读出放大器等），而一个SRAM存储单元大约需要六个晶体管。
- DRAM一般行列地址复用，SRAM行列地址独立。
- DRAM容量较SRAM大，但是读写速度不如SRAM
- SRAM用双稳态触发器来保存信息，而DRAM的是用电容，要定期刷新来保持。
- 由于SRAM成本高，目前SRAM基本上只用于CPU内部缓存，少量的网络服务器以及路由器。

SRAM 与 DRAM

	SRAM	DRAM
存储原理	双稳态触发器	电容电荷
每位晶体管数	6	1
相对访问时间	1×	10×
持续的？	是	否
敏感的？	否	是
容量	较小	较大
相对成本	100×	1×
地址复用封装	否	是
主要应用领域	高速缓存、高端路由器	主存、帧缓冲区

DDR SDRAM

- **同步DRAM** (Synchronous DRAM, SDRAM)：所有操作（读、写、刷新）都由系统时钟控制。该系统时钟与CPU的时钟速度同步。
- **双倍速率同步DRAM** (Double Data-rate Synchronous DRAM, DDR SDRAM)
 - 每个周期上升沿和下降沿都传输，因此它的实际数据传输频率是外部给它的时钟信号频率的两倍。
 - 设置一个多位的预取缓冲区，每次读取多个二进制位，然后可以分多次进行传输。
 - DDR的预取缓冲区是2位，每次读取2bit，然后分别利用该周期的上升沿和下降沿，分两次将2bit进行传输。因此DDR倍增系数为2。
 - 不同类型的DDR技术就是用预取缓冲区的大小来划分的：DDR（2位）、DDR2（4位）、DDR3（8位）。

DDR SDRAM

■ DDR4:

- 数据预取增加更多有困难，所以推出了Bank Group的设计，每个Bank Group可以独立读写数据。
- DDR4架构上采用了16n预取的Bank Group分组，包括两个Bank Group分组，每个Bank Group分组都有独立的激活、读取、写入和刷新操作。

6.3 数据校验码

- 数据在存取和传送过程中可能会发生错误，对数据采用某种编码，可以发现甚至确定出错位置。
- **数据校验码**：一种常用的能够发现错误甚至能够自动纠正错误的编码方法，又称之为“检错纠错编码”。
- 任何一种编码都由许多码字构成，任意两个合法码字之间**最少变化的二进制位数**，被称为数据校验码的**码距**。
- 例如，用四位二进制表示16种状态，则有16个不同的码字，此时码距为1，即两个码字之间最少仅有一个二进制位不同（如0000与0001之间）。这种编码没有检错能力，因为当某一个合法码字中有一位或几位出错，就变成为另一个合法码字。

数据校验码

- 数据校验码的实现原理：在编码中，除去合法的码字外，再加进一些非法码字，当某个合法码字出现错误时，就变成为非法码字。合理地安排非法码字的数量和编码规则，就能达到纠错的目的。
- 例如，若用4位二进制表示8个状态，其中只有8个码字是合法码字，而另8个码字为非法码字，此时码距可以为2。
- 码距大于等于2的数据校验码，开始具有检错的能力。码距越大，检错和纠错能力就越强，而且检错能力总是大于或等于纠错能力。
- 合理增大码距，能提高发现错误的能力，但会增加使用的二进制数，增加了数据存储的容量或数据传送的数量。

奇偶校验码

- **奇偶校验码**：一种最简单的数据校验码，码距等于2，可以检测出一位（或奇数位）错误，但不能确定出错的位置，也不能检测出偶数位错误。
- 实现方法：由若干位有效信息（如一个字节），再加上一个二进制位（校验位）组成校验码，然后根据校验码的奇偶性质进行校验。
- **奇偶校验码 (N+1位) = N位有效信息 + 1位校验位**
- **奇校验和偶校验**：校验位的取值（0或1）使整个校验码中“1”的个数为奇数或偶数。
- 因为1位出错的概率要比多位出错的概率高得多，所以奇偶校验码还是一种应用最广泛的校验方法，常用于存储器读、写检查或ASCII字符传送过程中的检查。

奇偶校验码

- **交叉校验：**计算机在进行大量字节（数据块）传送时，不仅每一个字节有一个奇偶校验位做横向校验，而且全部字节的同一位也设置一个奇偶校验位做纵向校验，这种横向、纵向同时校验的方法称为交叉校验。

例如，4 个字节组成的信息块，纵、横向均为偶校验，各校验位取值如下：

	A ₇	A ₆	A ₅	A ₄	A ₃	A ₂	A ₁	A ₀	横向校验位
第 1 字节	1	1	0	0	1	0	1	1	→1
第 2 字节	0	1	0	1	1	1	0	0	→0
第 3 字节	1	0	0	1	1	0	1	0	→0
第 4 字节	1	0	0	1	0	1	0	1	→0
	↓	↓	↓	↓	↓	↓	↓	↓	
纵向校验位	1	0	0	1	1	0	0	0	

身份证校验位计算方法

1. 将前面的身份证号码17位数分别乘以不同的系数。第 i 位对应的系数为 $[2^{(18-i)}] \bmod 11$ 。从第一位到第十七位的系数分别为：7 9 10 5 8 4 2 1 6 3 7 9 10 5 8 4 2；
2. 将这17位数字和系数相乘的结果相加；
3. 用加出来的和除以11，求余数；
4. 余数只可能有0 1 2 3 4 5 6 7 8 9 10这11个数字。其分别对应的最后一位身份证的号码为1 0 X 9 8 7 6 5 4 3 2；
5. 通过上面得知如果余数是2，就会在身份证的第18位数字上出现罗马数字的X。如果余数是10，身份证的最后一位号码就是2

海明校验码

Richard Hamming 于1950年提出，是一种**多重奇偶校验**，其基本思想：

- 在有效信息位中加入几个校验位形成海明码，并把海明码的每一个二进制位分配到几个奇偶校验组中。
- 当某一位出错后，就会引起有关的几个校验位的值发生变化，可以指出错误的位置（纠错）

假设校验位的个数为 k ，能表示 2^k 个信息，用其中的一个信息表示“没有错误”，其余 2^k-1 个信息指出错误发生在哪一位。因为错误也可能发生在校验位，所以只有 $r=2^k-1-k$ 个信息能用于纠正被传送数据的错误，即校验位的位数 k 与信息位的位数 r 应满足下列关系：

$$2^k \geq r+k+1$$

海明校验码

- 若海明码的最高位编号为 m ，最低为编号为1，海明码可表示为：

$$H_m H_{m-1} \dots H_2 H_1$$

- 海明码的编码规则：

- ① 位号数（1、2、3、……、 n ）为2的权值的那些位，即 $1(2^0)$ 、 $2(2^1)$ 、 $4(2^2)$ 、 $\dots 2^{(r-1)}$ 位，作为奇偶校验位，记作 P_1 、 P_2 、 $\dots P_r$ ，余下各位则为有效信息位。
- ② 海明码每一位 H_i 由多个校验位校验， i 等于校验它的各校验位的位号之和，即海明码的位号实质上是参与校验的各校验位权值之和。这样，校验的结果能正确反应出错的位号。

海明校验码的编码

- 一个字节由8位二进制位组成，即 $n=8$ ，若 $k=4$ ，海明码的总位数为12位，可表示为：

$H_{12} H_{11} \dots H_2 H_1$

- 4个校验位 $P_4 \sim P_1$ 对应的海明码位号应分别为： H_8 、 H_4 、 H_2 、 H_1 ，都满足 P_i 的位号等于 2^{i-1} 的关系，因此，有如下排列关系：

H_{12}	H_{11}	H_{10}	H_9	H_8	H_7	H_6	H_5	H_4	H_3	H_2	H_1
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
D_8	D_7	D_6	D_5	P_4	D_4	D_3	D_2	P_3	D_1	P_2	P_1

海明校验码的编码

- 各个信息位形成校验位 P_i ($i=1 \sim 4$) 的偶校验的结果为:

$$P_1 = D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7$$

$$P_2 = D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7$$

$$P_3 = D_2 \oplus D_3 \oplus D_4 \oplus D_8$$

$$P_4 = D_5 \oplus D_6 \oplus D_7 \oplus D_8$$

- 每一位信息位都均匀地出现在2~3个 P_i 值的形成关系中。当任一信息位发生变化时, 就会引起2~3个 P_i 值跟着变化, 即海明码的码距为3。

$H_{12} \ H_{11} \ H_{10} \ H_9 \ H_8 \ H_7 \ H_6 \ H_5 \ H_4 \ H_3 \ H_2 \ H_1$

$D_8 \ D_7 \ D_6 \ D_5 \ P_4 \ D_4 \ D_3 \ D_2 \ P_3 \ D_1 \ P_2 \ P_1$

海明校验码的校验

- 将接收到的海明码按如下关系进行偶校验，即：

$$S_1 = P_1 \oplus D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7$$

$$S_2 = P_2 \oplus D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7$$

$$S_3 = P_3 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_8$$

$$S_4 = P_4 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_8$$

- 校验结果 $S_4 \sim S_1$ 称为**指误字**，它能反映12位海明码的出错情况：
 - 当 $S_4 \sim S_1$ 为0000时，表明**无错**。
 - 当 $S_4 \sim S_1$ 中不为0，表明是某一校验位出错（包括信息位和校验位）**出错位由 $S_4 \sim S_1$ 指出**。
 - 例如 $S_4 \sim S_1 = 0011$ ，表示 H_3 (D_1) 出错，把 D_1 求反，即得到正确结果。这就是一位错的纠错。

海明校验码的扩展

- 8位数据，4位校验位的海明码，若有两位出错，将会如何？

假设D1和D3两位出错，

S1中包含D1，故 $S1=1$ ；

S2中同时包含D1，D3，故 $S2=0$ ；

S3中包含D3，故 $S3=1$ ；

S4中不包含D1，D3，故 $S4=0$ 。

即 $S4 \sim S1 = 0101 = 5$ ，指出D2出错，指示错误

海明校验码的扩展

- 能检测和自动校正一位错并能发现两位错的海明码，由于要保留一个校验位用于发现两位错，所以校验位的位数k与信息位的位数n应满足下列关系

$$2^{k-1} \geq r+k+1$$

校验位数k与信息位数r的对应关系

r值	最小k值	r值	最小k值
1~3	4	26~56	7
4~10	5	57~119	8
11~25	6		

海明校验码的扩展

■ 编码

- 一个字节由8位二进制位组成，即 $n=8$ ，若 $k=5$ ，海明码的总位数为13位，可表示为：

$H_{13} H_{12} \dots H_2 H_1$

- 5个校验位 $P_5 \sim P_1$ 对应的海明码位号应分别为： H_{13} 、 H_8 、 H_4 、 H_2 、 H_1 ，除 P_5 外，其余4位都满足 P_i 的位号等于 2^{i-1} 的关系， P_5 放在 H_{13} 上。

$P_5 D_8 D_7 D_6 D_5 P_4 D_4 D_3 D_2 P_3 D_1 P_2 P_1$

海明校验码的扩展

- 各个信息位形成校验位 P_i ($i=1 \sim 4$) 的偶校验的结果为:

$$P_1 = D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7$$

$$P_2 = D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7$$

$$P_3 = D_2 \oplus D_3 \oplus D_4 \oplus D_8$$

$$P_4 = D_5 \oplus D_6 \oplus D_7 \oplus D_8$$

- 另外新增一个总校验位 P_5 (即 H_{13}) 对所有其他的校验位和数字位提供偶校验

$$P_5 = H_1 \oplus H_2 \oplus H_3 \oplus \dots \oplus H_{12}$$

- 如果发生1位错和2位错, 通过 P_5 的校验能检测出来。
- 每一位信息位都至少出现在3个 P_i 值的形成关系中。当任一信息位发生变化时, 必将引起3个 P_i 值跟着变化, 即扩展海明码的码距为4。

海明校验码的扩展

■ 校验

- 将接收到的海明码按如下关系进行偶校验，即：

$$S_1 = P_1 \oplus D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7$$

$$S_2 = P_2 \oplus D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7$$

$$S_3 = P_3 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_8$$

$$S_4 = P_4 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_8$$

$$S_5 = P_5 \oplus H_1 \oplus H_2 \oplus H_3 \oplus \dots \oplus H_{12}$$

海明校验码的扩展

- 扩展海明码有以下关系成立

- ① 当所有位均没错时, $S_5=0$, $S=S_4S_3S_2S_1=0$

- ② 当出现一位错时, $S_5=1$

- $S=0$, 则 P_5 出错,

- $S \neq 0$, 则 S 指出是哪一位出错

- ③ 当两位出错时, $S_5=0$, $S \neq 0$ 。

- 另外注意:

- $S_5=0$, $S=0$, 有可能是多位错 (如4位错), 即 $S_5=0$, $S=0$, 有可能是偶数位错;

- $S_5=1$, 有可能是奇数位错(并不意味着只是1位错)。

海明校验码的扩展示例

■ 例：设有一个8位信息为10101100，试求海明编码的生成和校验过程。

1. 编码生成

检验位长度为5，按偶校验有：

$$P_1 = D_1 \oplus D_2 \oplus D_4 \oplus D_5 \oplus D_7 = 0 \oplus 0 \oplus 1 \oplus 0 \oplus 0 = 1$$

$$P_2 = D_1 \oplus D_3 \oplus D_4 \oplus D_6 \oplus D_7 = 0 \oplus 1 \oplus 1 \oplus 1 \oplus 0 = 1$$

$$P_3 = D_2 \oplus D_3 \oplus D_4 \oplus D_8 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$$

$$P_4 = D_5 \oplus D_6 \oplus D_7 \oplus D_8 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$$

$$P_5 = D_1 \oplus \dots \oplus D_8 \oplus P_1 \oplus \dots \oplus P_4 = 1$$

可得到用二进制表示的海明码为：

1 1 0 1 0 0 1 1 0 1 0 1 1

H_{13} H_{12} H_{11} H_{10} H_9 H_8 H_7 H_6 H_5 H_4 H_3 H_2 H_1

P_5 D_8 D_7 D_6 D_5 P_4 D_4 D_3 D_2 P_3 D_1 P_2 P_1

海明校验码的扩展示例

2. 校验

假设海明码传送后 H_{11} (D_7) 位发生了错误, 原来的码字变为:

1 1 1 1 0 0 1 1 0 1 0 1 1

出错

检错的过程很简单, 只要将接收到的码字重新进行偶校验:

$$S_1 = 1 \oplus 0 \oplus 0 \oplus 1 \oplus 0 \oplus 1 = 1$$

$$S_2 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 1 \oplus 1 = 1$$

$$S_3 = 1 \oplus 0 \oplus 1 \oplus 1 \oplus 1 = 0$$

$$S_4 = 0 \oplus 0 \oplus 1 \oplus 1 \oplus 1 = 1$$

$$S_5 = 1 \oplus 1 \oplus 1 \oplus 1 \oplus 0 \oplus 0 \oplus 1 \oplus 1 \oplus 0 \oplus 1 \oplus 0 \oplus 1 \oplus 1 = 1$$

- $S_5 = 1$, $S_4 S_3 S_2 S_1 = 1011$, 相应的十进制数是11, 指出 H_{11} 出错。现在 H_{11} 错成了“1”, 纠错就是将 H_{11} 位取反让它恢复为“0”。
- 纠正后为1 1 0 1 0 0 1 1 0 1 0 1 1, 原始数据为10101100